

课程笔记 SNN训练算法汇总综述

Tao

摘要

课程笔记 SNN训练算法汇总综述，涵盖了从传统的 BPTT/STBP 到最新的在线学习方法（OTTT、NDOT）以及局部学习规则（S-TLLR、TESS）。重点分析了每种方法的核心思想、数学推导、优缺点以及适用场景。通过对比不同算法在时间依赖性处理、梯度计算复杂度和生物可解释性方面的差异，全面总结了 SNN 训练领域的研究进展和未来方向。

PPT Week4_SNN Training.pptx

文章汇总

RTRL: A learning algorithm for continually running fully recurrent neural networks [1]; E-prop: A solution to the learning dilemma for recurrent networks of spiking neurons [2]; OTTT: Online Training Through Time for Spiking Neural Networks [3]; NDOT: NDOT: Neuronal Dynamics-based Online Training for Spiking Neural Networks [4]; S-TLLR: S-TLLR: STDP-inspired Temporal Local Learning Rule for Spiking Neural Networks [5]; TESS: TESS: A Scalable Temporally and Spatially Local Learning Rule for Spiking Neural Networks [6]

I. ELIGIBILITY TRACE: REPLACING BPTT MEMORY

Eligibility Trace: Replacing BPTT Memory

BPTT's Memory Wall:

- Unrolls time, stores every state — memory $\mathcal{O}(Tn)$.
- Learning must wait until the end — no online adaptation.

Eligibility Trace: A Smarter Way:

- Replace full history with a recursive trace e_{ji}^t .
- Gradients flow forward in time, not backward.
- Error signals meet historical traces in the present.

The Core Formula:

$$\Delta W_{ji} = \sum_t L_j^t \cdot e_{ji}^t \quad (1)$$

L : learning signal (what went wrong).

e : eligibility trace (what caused it).

【学生版讲稿】

BPTT 在处理时间序列时面临一个根本性的困境：它需要将所有时间步展开，存储每个神经元在每个时刻的状态，内存与时间步长 T 成正比。更严重的是，学习必须等待整个序列结束后才能开始，无法实现实时在线学习。

Eligibility Trace 提供了一种截然不同的思路。它不再试图存储完整的历史，而是用一个递归变量 e_{ji}^t 来压缩和总结历史信息。这个“资格迹”像是一个记忆痕迹，把过去的活动信息携带到现在。当误差信号 L_j^t 到达时，它只需要与这个已经累积好的痕迹相乘，就能得到完整的梯度信息。误差无需反向传播到过去——过去已经被“压缩”并带到了现在。

这一思想的核心数学表达是 $\Delta W_{ji} = \sum_t L_j^t \cdot e_{ji}^t$ 。 L_j^t 是顶向下学习信号（告诉网络哪里错了）， e_{ji}^t 是局部资格迹（告诉网络什么导致了错误）。这个简洁的公式构成了 **RTRL**、**e-prop**、**OTTT**、**NDOT**、**S-TLLR**、**TESS** 等所有在线学习算法的共同理论基础。

【自我学习笔记】

1. 问题的起源：BPTT 的时空展开

BPTT 的核心操作是将循环神经网络在时间维度上“展开”。每个时间步 t 变成前馈网络中的一层，每一层的神经元状态 $u^{(l)}[t]$ 都必须被存储，因为反向传播时这些状态被用来计算梯度。具体地，对于 L 层网络、 T 个时间步、每层 n 个神经元，需要存储的状态数为 $O(TLn)$ 。当 T 很大时，这个内存开销迅速超过硬件极限。

更根本的问题在于，反向传播中 $\frac{\partial L}{\partial u^{(l)}[t]}$ 的计算依赖于未来时刻 $t+1, t+2, \dots, T$ 的信息：

$$\frac{\partial L}{\partial u^{(l)}[t]} = \frac{\partial L}{\partial o^{(l)}[t]} \frac{\partial o^{(l)}[t]}{\partial u^{(l)}[t]} + \frac{\partial L}{\partial u^{(l)}[t+1]} \frac{\partial u^{(l)}[t+1]}{\partial u^{(l)}[t]}.$$

这种对未来信息的依赖意味着：必须等整个前向传播完成后才能开始反向传播，无法在线学习。

2. Eligibility Trace 的本质：用递归替代记忆

Eligibility Trace 的核心洞察是：我们不需要存储每一个历史时刻的完整状态，只需要维护一个递归变量 e_{ji}^t ，它是对历史的“压缩表示”。这个变量的物理类比是生物神经元中的分子痕迹——钙离子浓度、磷酸化酶活性等——它们随突触活动累积，随时间指数衰减，但不会立即消失。

这个递归变量的通式为：

$$e_{ji}^{t+1} = \alpha \cdot e_{ji}^t + \beta \cdot (\text{当前突触活动}),$$

其中 α 是衰减因子， β 是当前输入的贡献系数。具体形式取决于神经元模型和算法设计。

3. 统一数学形式的推导（完整版）

目标是计算损失 L 对权重 W_{ji} 的梯度。这个梯度告诉优化器： W_{ji} 增加一个微小量，损失会如何变化？

第一步：时间维度的链式法则展开。

$$\frac{\partial L}{\partial W_{ji}} = \sum_t \frac{\partial L}{\partial y_j[t]} \cdot \frac{\partial y_j[t]}{\partial u_j[t]} \cdot \frac{\partial u_j[t]}{\partial W_{ji}}. \quad (1)$$

式 (1) 中三个因子的含义：

- $\frac{\partial L}{\partial y_j[t]}$ ：损失对第 j 个神经元在 t 时刻输出的敏感度。如果 $y_j[t]$ 稍微增加，损失会变大还是变小？

- $\frac{\partial y_j[t]}{\partial u_j[t]}$: 输出对膜电位的导数。在 SNN 中, 这个量通常用代理梯度 ψ_j^t 来近似, 因为 $y_j[t] = H(u_j[t] - V_{th})$ 本身是不可微的。
- $\frac{\partial u_j[t]}{\partial W_{ji}}$: 膜电位对权重的敏感度。 W_{ji} 通过影响 t 时刻的输入电流来影响 $u_j[t]$ 。

第二步: $\frac{\partial u_j[t]}{\partial W_{ji}}$ 的展开。 膜电位 $u_j[t]$ 不仅依赖于当前时刻的输入, 还通过神经元的递推动力学依赖于所有过去时刻 $\tau \leq t$ 的输入。因此:

$$\frac{\partial u_j[t]}{\partial W_{ji}} = \sum_{\tau \leq t} \frac{\partial u_j[t]}{\partial u_j[\tau]} \cdot \frac{\partial u_j[\tau]}{\partial W_{ji}}. \quad (2)$$

式 (2) 中:

- $\frac{\partial u_j[t]}{\partial u_j[\tau]}$: 从 τ 到 t 的状态转移敏感度。它等于 $\prod_{i=\tau}^{t-1} \frac{\partial u_j[i+1]}{\partial u_j[i]}$, 包含了泄漏、脉冲发放、重置等所有中间动力学效应。
- $\frac{\partial u_j[\tau]}{\partial W_{ji}}$: τ 时刻膜电位对权重的直接敏感度。因为 $u_j[\tau]$ 通过 $W_{ji} \cdot z_i[\tau - 1]$ 直接依赖于 W_{ji} , 这个量等于突触前活动 $z_i[\tau - 1]$ (或 $s_i[\tau - 1]$, 取决于符号约定)。

第三步: 代入并重新组织求和。 将式 (2) 代入式 (1):

$$\frac{\partial L}{\partial W_{ji}} = \sum_t \underbrace{\left(\frac{\partial L}{\partial y_j[t]} \cdot \frac{\partial y_j[t]}{\partial u_j[t]} \right)}_{L_j^t} \cdot \underbrace{\left(\sum_{\tau \leq t} \frac{\partial u_j[t]}{\partial u_j[\tau]} \cdot \frac{\partial u_j[\tau]}{\partial W_{ji}} \right)}_{e_{ji}^t}. \quad (3)$$

定义两个核心量:

- **学习信号 L_j^t :** 第 j 个神经元在 t 时刻接收到的项向下误差信息。它告诉这个神经元: 你的输出对损失的影响是什么?
- **资格迹 e_{ji}^t :** 第 j 个神经元到第 i 个输入源的连接在 t 时刻的“记忆状态”。它总结了过去所有时刻中, 这个连接对当前膜电位的累积贡献。

最终得到:

$$\Delta W_{ji} = \sum_t L_j^t \cdot e_{ji}^t. \quad (4)$$

4. 资格迹的递归更新

从 e_{ji}^t 的定义可以推导出其递归更新规则。对于 $t + 1$ 时刻:

$$e_{ji}^{t+1} = \frac{\partial u_j[t+1]}{\partial u_j[t]} \cdot e_{ji}^t + \frac{\partial u_j[t+1]}{\partial W_{ji}}. \quad (5)$$

式 (5) 的直观解释:

- 第一项: 过去的贡献通过状态转移矩阵 $\frac{\partial u_j[t+1]}{\partial u_j[t]}$ 衰减后继续存在。在 LIF 神经元中, 这通常是泄漏因子 λ 。
- 第二项: 当前时刻 $t + 1$ 的突触前活动 $\frac{\partial u_j[t+1]}{\partial W_{ji}}$ 对资格迹的新增贡献。

这个递归形式是所有在线算法的核心工程基础。不同算法的主要差异体现在:

- 如何计算或近似 $\frac{\partial u_j[t+1]}{\partial u_j[t]}$ (固定衰减 vs 动态衰减)
- 如何计算或近似 $\frac{\partial u_j[t+1]}{\partial W_{ji}}$ (直接使用突触前活动 vs 使用滤波后的活动)
- 如何获得 L_j^t (精确反向传播 vs 随机反馈 vs 本地生成)

II. REPRESENT WORK—RTRL: REAL-TIME RECURRENT LEARNING

RTRL: Real-Time Recurrent Learning (1989)

The First Online Gradient Algorithm:

- Computes *exact* gradients without unrolling time.
- Maintains a sensitivity matrix $p_{ij}^k(t) = \partial y_k(t) / \partial w_{ij}$.
- Gradients flow *forward*, not backward.

The Core Idea:

- The effect of a weight on future outputs is propagated recursively.
- Weight change = error $\times$ accumulated sensitivity.

Key Formula:

$$p_{ij}^k(t+1) = f'_k(s_k(t)) \left[\sum_l w_{kl} p_{ij}^l(t) + \delta_{ik} z_j(t) \right] \quad (2)$$

The Price: Memory $\mathcal{O}(n^3)$ — exact but impractical.

【学生版讲稿】

RTRL 是 Williams 和 Zipser 于 1989 年发表的奠基性工作，它首次证明了梯度可以在不展开时间的情况下在线精确计算。RTRL 的核心思想是维护一个“敏感度矩阵” $p_{ij}^k(t)$ ，它追踪网络中的每一个输出 y_k 对每一个权重 w_{ij} 的敏感度。这个敏感度可以沿着时间前向递归更新： $p_{ij}^k(t+1)$ 由两部分构成——一部分是过去敏感度通过网络连接传播过来的贡献（ $\sum_l w_{kl} p_{ij}^l(t)$ ），另一部分是当前输入的直接贡献（ $\delta_{ik} z_j(t)$ ）。权重更新则简单地由当前误差乘以累计敏感度得到。

RTRL 的优点是数学上的精确性——它计算的是真正的梯度，没有任何近似。但代价是巨大的：全连接网络中需要存储 n^3 个敏感度值，时间上每步需要 $O(n^4)$ 次运算。这使得 RTRL 虽然理论优雅，但从未在大规模网络中得到实际应用。然而，RTRL 的工作证明了“在线梯度计算”是可能的，为后续所有在线学习算法奠定了理论基础。

【自我学习笔记】

1. 算法来源与历史定位

RTRL (Real-Time Recurrent Learning) 是 Williams 和 Zipser 于 1989 年发表在 Neural Computation 上的经典工作。在那个时代，BPTT 已经被提出 (Rumelhart, Hinton & Williams, 1986)，但其离线特性和内存需求促使研究者寻找替代方案。RTRL 的核心贡献是回答了这样一个问题：我们能否在网络的运行过程中同步计算梯度，而不需要等待整个序列结束？

2. 网络模型与符号定义

考虑一个有 n 个单元、 m 个外部输入的循环神经网络。符号约定如下：

- $\mathbf{y}(t) \in \mathbb{R}^n$: t 时刻所有单元的输出向量， $y_k(t)$ 是第 k 个单元的输出值。
- $\mathbf{x}(t) \in \mathbb{R}^m$: t 时刻的外部输入向量， $x_k(t)$ 是第 k 个输入的值。
- $\mathbf{z}(t) \in \mathbb{R}^{m+n}$: $\mathbf{y}(t)$ 和 $\mathbf{x}(t)$ 的拼接向量。 U 是单元索引集合， I 是输入索引集合。对于任意 k ，若 $k \in I$ 则 $z_k(t) = x_k(t)$ ，若 $k \in U$ 则 $z_k(t) = y_k(t)$ 。

- $\mathbf{W} \in \mathbb{R}^{n \times (m+n)}$: 权重矩阵。 w_{ij} 是第 i 个单元与第 j 个源之间的连接权重。注意 j 可以是单元索引 ($j \in U$) 或输入索引 ($j \in I$)。
- 偏置: 通过引入一个值恒为 1 的特殊输入 $x_{m+1} = 1$ 来实现。这样偏置权重就被统一到了权重矩阵中。

网络的前向动力学为:

$$s_k(t) = \sum_{l \in U \cup I} w_{kl} z_l(t), \quad y_k(t+1) = f_k(s_k(t)). \quad (6)$$

其中 $s_k(t)$ 是第 k 个单元的净输入 (加权和), f_k 是非线性激活函数 (如 logistic sigmoid)。注意: 输入 $x(t)$ 在 t 时刻进入网络, 但直到 $t+1$ 时刻才影响输出。

3. 敏感度矩阵的定义

定义敏感度变量:

$$p_{ij}^k(t) \triangleq \frac{\partial y_k(t)}{\partial w_{ij}}. \quad (7)$$

这个三阶张量 $p \in \mathbb{R}^{n \times (m+n) \times n}$ 的物理意义是: 第 k 个单元在 t 时刻的输出 $y_k(t)$ 对权重 w_{ij} 的偏导数。它是一个三阶量, 因为我们需要知道“哪个输出”对“哪个权重”的敏感度。

初始条件: $p_{ij}^k(t_0) = 0$, 即网络的初始状态与权重无关 (初始状态是固定的, 不由权重决定)。

4. 敏感度递归的完整推导

从网络动力学式 (6) 出发, 对 w_{ij} 求偏导:

$$\frac{\partial y_k(t+1)}{\partial w_{ij}} = f'_k(s_k(t)) \cdot \frac{\partial s_k(t)}{\partial w_{ij}}. \quad (8)$$

现在展开 $\frac{\partial s_k(t)}{\partial w_{ij}}$ 。 $s_k(t) = \sum_l w_{kl} z_l(t)$, 但 $z_l(t)$ 中包含了 $y_l(t)$ (当 $l \in U$) 和 $x_l(t)$ (当 $l \in I$)。 $y_l(t)$ 本身也依赖于权重 (通过过去时刻的影响), 而 $x_l(t)$ 是外部输入, 与权重无关。

因此:

$$\frac{\partial s_k(t)}{\partial w_{ij}} = \sum_{l \in U} w_{kl} \frac{\partial y_l(t)}{\partial w_{ij}} + \frac{\partial}{\partial w_{ij}} \left(\sum_{l \in U} w_{kl} y_l(t) \right)_{\text{显式}} + \frac{\partial}{\partial w_{ij}} \left(\sum_{l \in I} w_{kl} x_l(t) \right). \quad (9)$$

逐项分析式 (9):

- 第一项 $\sum_{l \in U} w_{kl} \frac{\partial y_l(t)}{\partial w_{ij}}$: 这是通过网络中其他单元的输出 $y_l(t)$ 传递过来的间接影响。因为 $y_l(t)$ 可能通过循环连接依赖于 w_{ij} 。
- 第二项: $\frac{\partial}{\partial w_{ij}} (\sum_{l \in U} w_{kl} y_l(t))$ 。这里需要区分两种情况:
 - 若 $k = i$ (即我们正在考虑第 i 个单元的输入权重), 且 $l = j$ (且 $j \in U$), 则 $\frac{\partial}{\partial w_{ij}} (w_{ij} y_j(t)) = y_j(t)$ 。
 - 否则为 0。

这可以统一写为 $\delta_{ik} \cdot [j \in U] \cdot y_j(t)$, 其中 δ_{ik} 是克罗内克函数, $[j \in U]$ 是指示函数。

- 第三项: $\frac{\partial}{\partial w_{ij}} (\sum_{l \in I} w_{kl} x_l(t))$ 。类似地, 只有当 $k = i$ 且 $j \in I$ 时非零, 贡献为 $x_j(t)$ 。

将第二项和第三项合并, 注意到 $z_j(t) = y_j(t)$ (若 $j \in U$) 或 $z_j(t) = x_j(t)$ (若 $j \in I$), 得到:

$$\frac{\partial s_k(t)}{\partial w_{ij}} = \sum_{l \in U} w_{kl} \frac{\partial y_l(t)}{\partial w_{ij}} + \delta_{ik} z_j(t). \quad (10)$$

将式 (10) 代入式 (8)，并利用敏感度矩阵的定义式 (7)：

$$p_{ij}^k(t+1) = f'_k(s_k(t)) \left(\sum_{l \in U} w_{kl} p_{ij}^l(t) + \delta_{ik} z_j(t) \right). \quad (11)$$

式 (11) 是 RTRL 的核心递归公式。它告诉我们： $t+1$ 时刻的敏感度 $p_{ij}^k(t+1)$ 可以从 t 时刻的敏感度 $p_{ij}^l(t)$ 通过权重矩阵向前传播一步得到，再加上当前输入 $z_j(t)$ 的直接贡献。

5. 权重更新规则

网络的瞬时误差定义为：

$$e_k(t) = d_k(t) - y_k(t), \quad (12)$$

其中 $d_k(t)$ 是 t 时刻的目标值。瞬时平方误差为：

$$J(t) = \frac{1}{2} \sum_{k \in U} [e_k(t)]^2. \quad (13)$$

总误差为所有时刻的累加： $J_{\text{total}} = \sum_t J(t)$ 。

梯度下降更新法则：

$$\Delta w_{ij}(t) = -\alpha \frac{\partial J(t)}{\partial w_{ij}} = \alpha \sum_{k \in U} e_k(t) p_{ij}^k(t). \quad (14)$$

式 (14) 的关键在于： $e_k(t)$ 是当前时刻的误差， $p_{ij}^k(t)$ 是累计到当前时刻的敏感度。两者相乘即可得到当前时刻的梯度贡献，无需等待未来信息。

6. Teacher Forcing 变体

当某些单元在特定时刻有目标值时（例如输出层），可以用目标值 $d_k(t)$ 替代实际输出 $y_k(t)$ 来驱动后续计算。这称为“教师强制”（Teacher Forcing）。在 Teacher Forcing 下， $\mathbf{z}(t)$ 的定义变为：

$$z_k(t) = \begin{cases} x_k(t), & k \in I \quad (\text{外部输入}) \\ d_k(t), & k \in T(t) \quad (\text{该时刻有目标值}) \\ y_k(t), & k \in U - T(t) \quad (\text{无目标值}) \end{cases} \quad (15)$$

其中 $T(t)$ 是 t 时刻有目标值的单元集合。

此时，敏感度递归变为：

$$p_{ij}^k(t+1) = f'_k(s_k(t)) \left(\sum_{l \in U - T(t)} w_{kl} p_{ij}^l(t) + \delta_{ik} z_j(t) \right). \quad (16)$$

注意求和只覆盖 $U - T(t)$ ——教师强制单元的敏感度被置为零，因为它们不再参与网络的后续动力学。

7. 计算复杂度分析

RTRL 的计算开销主要来自三个方面：

- **内存：**三阶张量 p_{ij}^k 的维度为 $(n \times (m+n) \times n)$ 。在全连接网络中 $m \approx n$ ，因此内存复杂度为 $\mathcal{O}(n^3)$ 。

- **时间（每步）**：更新所有 p_{ij}^k 需要 $\mathcal{O}(n^4)$ 次运算。这是因为对于每个权重 w_{ij} ($\mathcal{O}(n^2)$ 个)，需要更新 n 个 k ，每个更新需要对 l 求和 ($\mathcal{O}(n)$)。
- **通信**：每个权重需要访问全局权重矩阵 $\mathbf{W}$ 和全局误差向量 $\mathbf{e}$ ，通信开销巨大。这也是 RTRL 虽然理论优美但从未被实际采用的根本原因。

8. 历史贡献与后续影响

尽管 RTRL 不实用，它首次证明了“精确梯度可以在不存储历史的情况下在线计算”这一可能性。这个证明为后续所有在线学习算法提供了理论基础：

- **e-prop**：本质上是在 RTRL 框架下引入生物合理性约束（代理梯度、慢变量）和近似（随机反馈）。
- **OTTT/NDOT**：在 RTRL 框架下做进一步近似（忽略某些项），将复杂度从 $\mathcal{O}(n^3)$ 降至 $\mathcal{O}(n)$ 。
- **S-TLLR/TESS**：在 RTRL 框架下引入额外的生物机制（STDP 非因果项、本地学习信号生成）。

III. REPRESENT WORK—E-PROP

e-Prop: Eligibility Propagation (2020)

Bridging Neuroscience and Machine Learning:

- The brain doesn't use BPTT, yet learns long-term dependencies.
- e-prop's insight: let error signals meet historical traces in the present.

How It Works:

- Eligibility traces carry the past forward.
- Learning signals (errors) arrive from the top.
- Weight update = learning signal $\times$ eligibility trace.

The Secret: Slow Neurons (ALIF):

- Adaptive firing thresholds create long-lasting traces ($\tau_a \approx 1000\text{ms}$).
- These are "highways into the future" — early inputs meet late errors.

Key Formula:

$$\frac{dE}{dW_{ji}} = \sum_t L_j^t \cdot e_{ji}^t, \quad e_{ji}^t = \psi_j^t(z_i^{t-1} - \beta \epsilon_{ji,a}^t) \quad (3)$$

Three Variants: Symmetric (weight sharing), Random (biologically plausible), Adaptive.

【学生版讲稿】

e-Prop 是 Bellec 等人于 2020 年发表在 Nature Communications 上的工作，它首次将 Eligibility Trace 与生物合理性深度结合，回答了神经科学中的一个核心问题：大脑如何在不用 BPTT 的情况下学习长期依赖？e-Prop 的核心洞察是：不需要让误差信号反向传播到过去，而是让资格迹把过去的信息带到现在。当误差信号从网络顶层传到某个神经元时，它只需要与这个神经元已经累积好的资格迹相乘，就能完成信用分配。

e-Prop 的关键创新在于引入了 ALIF（自适应 LIF）神经元。这种神经元的发放阈值会随自身的发放历史缓慢变化（时间常数可达数百毫秒到数秒），这使得资格迹能在长时间尺度上保持活性。

这些慢变量充当了“通往未来的高速公路”——早期的输入信息可以被携带到晚期，与晚期才出现的误差信号相遇。e-Prop 有三种变体：对称 e-prop（需要权重转置，精度最高）、随机 e-prop（用固定随机反馈替代反向传播，最生物合理）、自适应 e-prop（折中方案）。实验表明，随机 e-prop 在 TIMIT 音素识别等任务上逼近 BPTT 的性能。

【自我学习笔记】

1. 算法定位与动机

e-Prop (Bellec et al., 2020, Nature Communications) 是第一个将 Eligibility Trace 与生物合理性深度结合的 SNN 训练算法。其核心动机来自两个神经科学观察：

- 1) 神经元在分子层面维持着过去活动的“痕迹”（如钙离子浓度、CaMKII 磷酸化），这些痕迹随时间指数衰减但不会立即消失。
- 2) 大脑中存在丰富的顶向下信号（如多巴胺、乙酰胆碱），这些信号传递的是行为结果而非精确的误差梯度。

e-Prop 试图回答：如何将这两类信号结合起来，实现生物合理的梯度下降学习？

2. BPTT 的重新因子化

e-Prop 的出发点是 BPTT 的梯度表达式。对于循环神经网络，BPTT 的梯度可以写成：

$$\frac{dE}{dW_{ji}} = \sum_{t'} \frac{dE}{dh_j^{t'}} \cdot \frac{\partial h_j^{t'}}{\partial W_{ji}}. \quad (15)$$

式 (15) 中：

- $\frac{dE}{dh_j^{t'}}$ ：损失 E 对隐藏状态 $\mathbf{h}_j^{t'}$ 的“总”导数。注意这里的 d 是总导数，它包含了 $\mathbf{h}_j^{t'}$ 通过后续所有时间步对损失的间接影响。
- $\frac{\partial h_j^{t'}}{\partial W_{ji}}$ ：隐藏状态对权重的直接偏导。

e-Prop 的关键操作是递归展开 $\frac{dE}{dh_j^{t'}}$ 。应用链式法则于计算图的节点 $\mathbf{h}_j^{t'}$ ：

$$\frac{dE}{dh_j^{t'}} = \frac{dE}{dz_j^{t'}} \cdot \frac{\partial z_j^{t'}}{\partial h_j^{t'}} + \frac{dE}{dh_j^{t'+1}} \cdot \frac{\partial h_j^{t'+1}}{\partial h_j^{t'}}. \quad (17)$$

式 (17) 的直观解释：

- 第一项： $\mathbf{h}_j^{t'}$ 通过直接影响 t' 时刻的输出 $z_j^{t'}$ 来影响损失。这是“直接路径”。
- 第二项： $\mathbf{h}_j^{t'}$ 通过影响下一时刻的隐藏状态 $\mathbf{h}_j^{t'+1}$ 来间接影响损失。这是“间接路径”，需要递归处理。

递归终止条件： $\frac{dE}{dh_j^{T+1}} = 0$ ，因为 T 是最后一个时刻。

将递归展开代入式 (15)，并交换求和顺序：

$$\frac{dE}{dW_{ji}} = \sum_t \frac{dE}{dz_j^t} \cdot \underbrace{\left(\frac{\partial z_j^t}{\partial h_j^t} \sum_{\tau \leq t} \frac{\partial h_j^t}{\partial h_j^\tau} \cdot \frac{\partial h_j^\tau}{\partial W_{ji}} \right)}_{e_{ji}^t}. \quad (13)$$

这里 $\frac{\partial h_j^t}{\partial h_j^\tau} = \prod_{i=\tau}^{t-1} \frac{\partial h_j^{i+1}}{\partial h_j^i}$ 。

3. 资格迹的定义

由式 (13)，e-Prop 的资格迹定义为：

$$e_{ji}^t \triangleq \frac{\partial z_j^t}{\partial \mathbf{h}_j^t} \sum_{\tau \leq t} \frac{\partial \mathbf{h}_j^t}{\partial \mathbf{h}_j^\tau} \cdot \frac{\partial \mathbf{h}_j^\tau}{\partial W_{ji}}. \quad (18)$$

注意：

- e_{ji}^t 只依赖于神经元 j 和 i 的局部信息 ($\mathbf{h}_j$ 、 W_{ji} 等)，不依赖于网络其他部分。
- 它是“局部”计算的，可以在前向传播过程中递归更新。

资格迹 e_{ji}^t 的递归更新形式为：

$$\epsilon_{ji}^t = \frac{\partial \mathbf{h}_j^t}{\partial \mathbf{h}_j^{t-1}} \cdot \epsilon_{ji}^{t-1} + \frac{\partial \mathbf{h}_j^t}{\partial W_{ji}}, \quad e_{ji}^t = \frac{\partial z_j^t}{\partial \mathbf{h}_j^t} \cdot \epsilon_{ji}^t. \quad (14)$$

其中 ϵ_{ji}^t 被称为“eligibility vector”，它与 $\mathbf{h}_j^t$ 具有相同的维度。

4. LIF 神经元的资格迹

对于标准 LIF 神经元（无自适应阈值）， $\mathbf{h}_j^t = v_j^t$ （一维膜电位）。从式 (14) 可得：

$$\frac{\partial \mathbf{h}_j^{t+1}}{\partial \mathbf{h}_j^t} = \alpha, \quad \frac{\partial \mathbf{h}_j^{t+1}}{\partial W_{ji}} = z_i^t.$$

其中 $\alpha = e^{-\Delta t / \tau_m}$ 是泄漏因子。因此：

$$\epsilon_{ji}^{t+1} = \alpha \epsilon_{ji}^t + z_i^t.$$

这是一个低通滤波器的递归形式，其解为 $\epsilon_{ji}^t = \mathcal{F}_\alpha(z_i^{t-1}) = \bar{z}_i^{t-1}$ （突触前脉冲的指数加权平均）。最终：

$$e_{ji}^t = \psi_j^t \cdot \bar{z}_i^{t-1}. \quad (23)$$

其中 $\psi_j^t = \frac{\partial z_j^t}{\partial \mathbf{h}_j^t}$ 是代理梯度。

5. ALIF 神经元的资格迹（完整推导）

ALIF (Adaptive LIF) 神经元在 LIF 基础上增加了自适应阈值。其完整动力学：

$$A_j^t = v_{th} + \beta a_j^t, \quad (8)$$

$$z_j^t = H(v_j^t - A_j^t), \quad (9)$$

$$v_j^{t+1} = \alpha v_j^t + \sum_i W_{ji}^{rec} z_i^t - v_{th} z_j^t, \quad (6)$$

$$a_j^{t+1} = \rho a_j^t + z_j^t. \quad (10)$$

参数解释：

- v_j^t ：膜电位（快速变量）。
- a_j^t ：适应变量（慢变量），控制阈值的浮动。
- β ：适应强度（通常约 0.07）。
- $\rho = e^{-\Delta t / \tau_a}$ ：适应变量的衰减因子， τ_a 通常为数百毫秒到数秒。

ALIF 的隐藏状态是二维的： $\mathbf{h}_j^t = [v_j^t, a_j^t]$ 。因此资格迹也是二维的： $\epsilon_{ji}^t = [\epsilon_{ji,v}^t, \epsilon_{ji,a}^t]$ 。

对于 v 分量: $\frac{\partial v_j^{t+1}}{\partial v_j^t} = \alpha$, $\frac{\partial v_j^{t+1}}{\partial W_{ji}} = z_i^t$, 因此:

$$\epsilon_{ji,v}^{t+1} = \alpha \epsilon_{ji,v}^t + z_i^t.$$

其解为 $\epsilon_{ji,v}^t = \bar{z}_i^{t-1}$ (同 LIF)。

对于 a 分量, 需要更细致的推导。首先, $z_j^t = H(v_j^t - v_{th} - \beta a_j^t)$ 对 v_j^t 和 a_j^t 的偏导通过代理梯度 ψ_j^t 给出:

$$\frac{\partial z_j^t}{\partial v_j^t} = \psi_j^t, \quad \frac{\partial z_j^t}{\partial a_j^t} = -\beta \psi_j^t.$$

从 $a_j^{t+1} = \rho a_j^t + z_j^t$ 对 W_{ji} 求导:

$$\epsilon_{ji,a}^{t+1} = \rho \epsilon_{ji,a}^t + \frac{\partial z_j^t}{\partial W_{ji}}.$$

而 $\frac{\partial z_j^t}{\partial W_{ji}} = \psi_j^t \frac{\partial (v_j^t - \beta a_j^t)}{\partial W_{ji}} = \psi_j^t (\epsilon_{ji,v}^t - \beta \epsilon_{ji,a}^t)$ 。代入 $\epsilon_{ji,v}^t = \bar{z}_i^{t-1}$:

$$\epsilon_{ji,a}^{t+1} = \psi_j^t \bar{z}_i^{t-1} + (\rho - \psi_j^t \beta) \epsilon_{ji,a}^t. \quad (24)$$

最终资格迹:

$$e_{ji}^t = \psi_j^t (\bar{z}_i^{t-1} - \beta \epsilon_{ji,a}^t). \quad (25)$$

6. 学习信号的近似

理想情况下, $L_j^t = \frac{dE}{dz_j^t}$ 是总导数。但在线计算中无法获得, 因此 e-Prop 用偏导数近似:

$$L_j^t = \frac{\partial E}{\partial z_j^t} \approx \sum_k B_{jk} (y_k^t - y_k^{t-1}). \quad (4)$$

其中 B_{jk} 是反馈权重。三种变体:

- 对称: $B_{jk} = W_{kj}^{out}$, 需要权重转置。
- 随机: B_{jk} 固定随机, 最生物合理。
- 自适应: B_{jk} 通过局部可塑性学习。

IV. REPRESENT WORK—OTTT (ONLINE TRAINING THROUGH TIME)

OTTT: Online Training Through Time (2022)

The Problem: Previous online algorithms still need $\mathcal{O}(n^2)$ memory.

OTTT's Answer: Track only presynaptic activity — $\mathcal{O}(n)$ memory.

The Key Observation — What to Drop in BPTT:

- BPTT's temporal dependency has two paths:
 - Direct path: $\frac{\partial \mathbf{u}^{l+1}[i+1]}{\partial \mathbf{u}^{l+1}[i]} = \lambda \mathbf{I}$ (leakage)
 - Spike path: $\frac{\partial \mathbf{u}^{l+1}[i+1]}{\partial \mathbf{s}^{l+1}[i]} \cdot \frac{\partial \mathbf{s}^{l+1}[i]}{\partial \mathbf{u}^{l+1}[i]}$ (reset via spikes)
- OTTT drops the spike path because $\frac{\partial \mathbf{s}}{\partial \mathbf{u}}$ (derivative of Heaviside) is 0 almost everywhere.
- Result: $\prod(\lambda + 0) = \lambda^{t-\tau}$ — a simple exponential decay!

Core Update:

$$\hat{\mathbf{a}}^l[t+1] = \lambda \hat{\mathbf{a}}^l[t] + \mathbf{s}^l[t+1], \quad \nabla_{\mathbf{w}^l} L[t] = \mathbf{g}_{\mathbf{u}^{l+1}}[t] \cdot (\hat{\mathbf{a}}^l[t])^\top \quad (4)$$

Why It Works: Memory $\mathcal{O}(n)$ — independent of T . Theoretically proven descent direction.

【学生版讲稿】

OTTT 是 Xiao 等人于 2022 年发表在 NeurIPS 上的工作，它解决了在线学习算法的一个关键瓶颈：虽然 RTRL 和 e-prop 实现了时间局部性，但它们的空间复杂度仍然较高（RTRL 是 $\mathcal{O}(n^3)$ ，e-prop 是 $\mathcal{O}(n^2)$ ）。OTTT 的目标是将内存降到 $\mathcal{O}(n)$ ，同时在大规模任务上验证有效性。

OTTT 的核心洞察来自对 BPTT 中时间依赖项的仔细审视。在 BPTT 的梯度公式中，有一个关键的连乘项：

$$\prod_{i=\tau}^{t-1} \left(\frac{\partial \mathbf{u}^{l+1}[i+1]}{\partial \mathbf{u}^{l+1}[i]} + \frac{\partial \mathbf{u}^{l+1}[i+1]}{\partial \mathbf{s}^{l+1}[i]} \frac{\partial \mathbf{s}^{l+1}[i]}{\partial \mathbf{u}^{l+1}[i]} \right).$$

这个连乘项包含两条路径：

- **直接路径：** $\frac{\partial \mathbf{u}^{l+1}[i+1]}{\partial \mathbf{u}^{l+1}[i]}$ ，膜电位通过泄漏直接衰减，在 LIF 中等于 $\lambda \mathbf{I}$ （纯指数衰减）。
- **脉冲路径：** $\frac{\partial \mathbf{u}^{l+1}[i+1]}{\partial \mathbf{s}^{l+1}[i]} \cdot \frac{\partial \mathbf{s}^{l+1}[i]}{\partial \mathbf{u}^{l+1}[i]}$ ，神经元脉冲发放后通过复位机制影响后续膜电位。

OTTT 的关键贡献是注意到： $\frac{\partial \mathbf{s}}{\partial \mathbf{u}}$ 是 Heaviside 阶跃函数的导数，几乎处处为 $\mathbf{0}$ （除了在阈值处的一个不可测点）。因此，在整个时间依赖传播中，脉冲路径的贡献可以忽略不计。于是连乘项简化为：

$$\prod_{i=\tau}^{t-1} (\lambda \mathbf{I} + \mathbf{0}) = \lambda^{t-\tau} \mathbf{I}.$$

就是纯粹的指数衰减！

这个化简的巨大意义在于：BPTT 中原本需要存储整个时间历史才能计算的连乘项，现在只需要知道时间差 $t - \tau$ ，用 $\lambda^{t-\tau}$ 就能直接算出来。于是可以定义 $\hat{\mathbf{a}}^l[t] = \sum_{\tau \leq t} \lambda^{t-\tau} \mathbf{s}^l[\tau]$ ，它在前向传播中递归更新：

$$\hat{\mathbf{a}}^l[t+1] = \lambda \hat{\mathbf{a}}^l[t] + \mathbf{s}^l[t+1],$$

只需要存储一个标量值。每步的梯度就是空间梯度 $\mathbf{g}$ 乘以这个时间迹：

$$\nabla_{\mathbf{w}^l} L[t] = \mathbf{g}_{\mathbf{u}^{l+1}}[t] \cdot (\hat{\mathbf{a}}^l[t])^\top.$$

最终内存从 $\mathcal{O}(n^2)$ 降到了 $\mathcal{O}(n)$ 。OTTT 的另一个贡献是从理论上证明了即使做了这个近似，其梯度仍然是有效的下降方向。

【自我学习笔记】

1. 算法定位与动机

OTTT (Xiao et al., 2022, NeurIPS) 是第一个将在线学习内存降到 $\mathcal{O}(n)$ 的算法，并在 ImageNet 等大规模任务上验证了有效性。它的核心贡献是回答了一个问题：在 BPTT 的复杂时间依赖中，哪一部分是真正重要的？哪一部分可以安全地忽略？

2. BPTT 梯度中的连乘项

对于从层 l 到层 $l+1$ 的权重 $\mathbf{W}^l$ ，BPTT 的梯度为：

$$\frac{\partial L}{\partial \mathbf{W}^l} = \sum_{t=1}^T \mathbf{g}_{\mathbf{u}^{l+1}}[t] \left(\frac{\partial \mathbf{u}^{l+1}[t]}{\partial \mathbf{W}^l} + \sum_{\tau < t} \mathbf{T}_{\tau,t} \cdot \frac{\partial \mathbf{u}^{l+1}[\tau]}{\partial \mathbf{W}^l} \right), \quad (3)$$

其中 $\mathbf{g}_{\mathbf{u}^{l+1}}[t] = \left(\frac{\partial L}{\partial \mathbf{s}^{l+1}[t]} \frac{\partial \mathbf{s}^{l+1}[t]}{\partial \mathbf{u}^{l+1}[t]} \right)^\top$ 是 t 时刻的空间梯度， $\mathbf{T}_{\tau,t}$ 是时间依赖的核心连乘项：

$$\mathbf{T}_{\tau,t} = \prod_{i=\tau}^{t-1} \left(\frac{\partial \mathbf{u}^{l+1}[i+1]}{\partial \mathbf{u}^{l+1}[i]} + \frac{\partial \mathbf{u}^{l+1}[i+1]}{\partial \mathbf{s}^{l+1}[i]} \frac{\partial \mathbf{s}^{l+1}[i]}{\partial \mathbf{u}^{l+1}[i]} \right). \quad (4)$$

3. 连乘项中两项的含义

连乘中的每一项都包含两个部分，对应两条不同的时间传播路径：

路径 1 — 直接衰减路径 (Direct Path)：

$$\frac{\partial \mathbf{u}^{l+1}[i+1]}{\partial \mathbf{u}^{l+1}[i]} = \lambda \mathbf{I}.$$

这是膜电位自身的动态演化。LIF 神经元的离散动力学为：

$$\mathbf{u}^{l+1}[i+1] = \lambda \mathbf{u}^{l+1}[i] + \mathbf{W}^l \mathbf{s}^l[i+1] - V_{th} \mathbf{s}^{l+1}[i],$$

其中 λ 是泄漏常数（通常取 0.5）。前一项 $\lambda \mathbf{u}^{l+1}[i]$ 对 $\mathbf{u}^{l+1}[i]$ 的偏导就是 $\lambda \mathbf{I}$ 。这条路径的物理含义是：膜电位信息按 λ 指数衰减地向前传递，没有任何信息损失以外的复杂效应。

路径 2 — 脉冲复位路径 (Spike/Reset Path)：

$$\frac{\partial \mathbf{u}^{l+1}[i+1]}{\partial \mathbf{s}^{l+1}[i]} \cdot \frac{\partial \mathbf{s}^{l+1}[i]}{\partial \mathbf{u}^{l+1}[i]}.$$

这条路径的物理含义是：

- $\frac{\partial \mathbf{s}^{l+1}[i]}{\partial \mathbf{u}^{l+1}[i]}$ ： i 时刻的膜电位是否达到阈值从而发放脉冲。这是 Heaviside 函数的导数。
- $\frac{\partial \mathbf{u}^{l+1}[i+1]}{\partial \mathbf{s}^{l+1}[i]}$ ： i 时刻的脉冲通过复位机制（减去 V_{th} ）影响 $i+1$ 时刻的膜电位。在 LIF 中，这个导数等于 $-V_{th}$ 。

这条路径是 BPTT 中处理脉冲发放对后续时间影响的唯一通道——它把“是否发放脉冲”这个离散事件纳入了梯度传播的连续框架。

4. OTTT 的关键观察

OTTT 观察到：

$$\frac{\partial \mathbf{s}^{l+1}[i]}{\partial \mathbf{u}^{l+1}[i]} = \frac{\partial}{\partial \mathbf{u}} H(\mathbf{u} - V_{th}).$$

这是 Heaviside 阶跃函数的导数。在分布意义下，它是 $\delta(\mathbf{u} - V_{th})$ （狄拉克函数）——除了在阈值点 $\mathbf{u} = V_{th}$ 处为无穷大外，其余地方全部为 0。在实际训练中，BP 使用代理梯度（surrogate gradient）来近似这个导数以提供有意义的梯度信息。但 OTTT 指出：在时间依赖的连乘中，这个项可以被安全地忽略，因为它在几乎所有时间步和所有神经元上都是 0。

这一观察与许多 BPTT with SG 实现中“detach reset”（将复位操作从计算图中分离）的做法完全一致——这些实现实际上已经隐式地做了同样的近似。区别在于，OTTT 明确指出了这个近似的合理性并给出了理论证明。

5. 化简后的连乘项

忽略脉冲路径后，连乘项简化为：

$$\mathbf{T}_{\tau,t} = \prod_{i=\tau}^{t-1} \frac{\partial \mathbf{u}^{l+1}[i+1]}{\partial \mathbf{u}^{l+1}[i]} = \prod_{i=\tau}^{t-1} \lambda \mathbf{I} = \lambda^{t-\tau} \mathbf{I}. \quad (5)$$

这个化简是 OTTT 整个算法的数学基石。它意味着：

- BPTT 中最复杂的部分——需要存储整个时间历史的连乘项——变成了一个仅依赖时间差 $t - \tau$ 的简单指数衰减。
- 这个衰减与神经元的具体状态无关，只与时间间隔有关。
- 任何时刻 t 的梯度计算，只需要知道当前时刻的突触前活动 $\mathbf{s}^l[t]$ 和之前所有时刻的指数加权

和。

6. 代入得到 OTTT 梯度

将 $\mathbf{T}_{\tau,t} = \lambda^{t-\tau} \mathbf{I}$ 代入式 (3)：

$$\frac{\partial L}{\partial \mathbf{W}^l} = \sum_{t=1}^T \mathbf{g}_{\mathbf{u}^{l+1}}[t] \left(\frac{\partial \mathbf{u}^{l+1}[t]}{\partial \mathbf{W}^l} + \sum_{\tau < t} \lambda^{t-\tau} \frac{\partial \mathbf{u}^{l+1}[\tau]}{\partial \mathbf{W}^l} \right). \quad (6)$$

注意到 $\frac{\partial \mathbf{u}^{l+1}[\tau]}{\partial \mathbf{W}^l} = \mathbf{s}^l[\tau]$ （突触前活动在第 τ 时刻的值），合并 $\tau \leq t$ 的项：

$$\frac{\partial L}{\partial \mathbf{W}^l} = \sum_{t=1}^T \mathbf{g}_{\mathbf{u}^{l+1}}[t] \left(\sum_{\tau \leq t} \lambda^{t-\tau} \mathbf{s}^l[\tau] \right)^\top. \quad (7)$$

7. 迹的递归定义

定义指数加权历史迹：

$$\hat{\mathbf{a}}^l[t] \triangleq \sum_{\tau \leq t} \lambda^{t-\tau} \mathbf{s}^l[\tau]. \quad (8)$$

这个量的物理含义是：到 t 时刻为止，层 l 所有脉冲活动的指数加权和——越近的脉冲权重越大，越远的脉冲权重以 λ 为底指数衰减。

由定义可推导出递归更新公式：

$$\hat{\mathbf{a}}^l[t+1] = \lambda \hat{\mathbf{a}}^l[t] + \mathbf{s}^l[t+1]. \quad (9)$$

因此，每步的梯度可以独立计算：

$$\boxed{\nabla_{\mathbf{W}^l} L[t] = \mathbf{g}_{\mathbf{u}^{l+1}}[t] \cdot (\hat{\mathbf{a}}^l[t])^\top}. \quad (10)$$

8. 为什么 OTTT 能降到 $\mathcal{O}(n)$ 内存

- 存储需求：只需要存储 $\hat{\mathbf{a}}^l[t]$ ——每个神经元一个标量值，共 $\mathcal{O}(n)$ 。注意这里的 n 是每层神经元数，与时间步长 T 完全无关。
- 不需要存储：
 - 历史脉冲序列 $\{\mathbf{s}^l[1], \mathbf{s}^l[2], \dots, \mathbf{s}^l[t]\}$ 。
 - 任何时间维度的中间梯度 $\frac{\partial \mathbf{u}}{\partial \mathbf{u}}$ 。

– 任何 $\frac{\partial \mathbf{s}}{\partial \mathbf{u}}$ 的历史值。

- 与 T 无关：无论序列有多长，内存恒为 $\mathcal{O}(n)$ 。相比之下，BPTT 的内存为 $\mathcal{O}(Tn)$ ，e-prop 为 $\mathcal{O}(n^2)$ 。

这个 $\mathcal{O}(n)$ 内存的实现前提正是“忽略脉冲路径”这个关键近似。如果保留脉冲路径，就需要存储 $\frac{\partial \mathbf{s}}{\partial \mathbf{u}}$ 的历史值（或者等价地，存储脉冲发放的确切时间），内存就会回到 $\mathcal{O}(n^2)$ 或更高。

9. 理论保证

OTTT 的另一个重要贡献是给出了理论保证。定理 1 的核心结论是：

$$\langle \nabla_{\text{OTTT}} L, \nabla_{\text{SR}} L \rangle > 0. \quad (11)$$

其中 $\nabla_{\text{SR}} L$ 是“脉冲表示法”（Spike Representation）的梯度。该方法通过将 SNN 的脉冲发放率映射到等效 ANN 的激活值，已被证明能提供有效的下降方向。

证明思路：OTTT 梯度与 $\nabla_{\text{SR}} L$ 的差异可以被控制。具体地，在加权发放率 $\mathbf{a}^l[t]$ 向 $\mathbf{a}^l[T]$ 收敛且收敛误差足够小的条件下：

$$\|\nabla_{\text{OTTT}} L - \nabla_{\text{SR}} L\| < \|\nabla_{\text{SR}} L\|,$$

从而保证内积为正。这证明 OTTT 的近似不会破坏优化的收敛性。

10. 关键贡献总结

- 1) 明确了 BPTT 中可忽略的项： $\frac{\partial \mathbf{s}}{\partial \mathbf{u}}$ （Heaviside 导数）在时间依赖连乘中几乎处处为 0。
- 2) 将连乘简化为指数衰减： $\prod \lambda = \lambda^{t-\tau}$ ，使得时间依赖仅取决于时间差。
- 3) 用递归迹替代历史存储： $\hat{\mathbf{a}}^l[t]$ 递归更新，内存 $\mathcal{O}(n)$ 与 T 无关。
- 4) 理论证明：近似梯度仍然是有效的下降方向。

【自我学习笔记】

1. 算法定位与动机

OTTT (Xiao et al., 2022, NeurIPS) 是第一个将在线学习内存降到 $\mathcal{O}(n)$ 的算法。在此之前，RTRL 需要 $\mathcal{O}(n^3)$ ，e-prop 需要 $\mathcal{O}(n^2)$ ，而 OTTT 通过巧妙的数学简化将内存降到了与权重数量无关的级别。

2. BPTT 梯度回顾

BPTT 的梯度表达式为：

$$\frac{\partial L}{\partial \mathbf{W}^l} = \sum_{t=1}^T \frac{\partial L}{\partial \mathbf{s}^{l+1}[t]} \frac{\partial \mathbf{s}^{l+1}[t]}{\partial \mathbf{u}^{l+1}[t]} \left[\frac{\partial \mathbf{u}^{l+1}[t]}{\partial \mathbf{W}^l} + \sum_{\tau < t} \left(\prod_{i=\tau}^{t-1} \left(\frac{\partial \mathbf{u}^{l+1}[i+1]}{\partial \mathbf{u}^{l+1}[i]} + \frac{\partial \mathbf{u}^{l+1}[i+1]}{\partial \mathbf{s}^{l+1}[i]} \frac{\partial \mathbf{s}^{l+1}[i]}{\partial \mathbf{u}^{l+1}[i]} \right) \right) \frac{\partial \mathbf{u}^{l+1}[\tau]}{\partial \mathbf{W}^l} \right]. \quad (3)$$

这个公式的复杂度来自连乘项 $\prod_{i=\tau}^{t-1}(\dots)$ ，它需要存储所有中间时间步的 $\frac{\partial \mathbf{u}}{\partial \mathbf{u}}$ 和 $\frac{\partial \mathbf{u}}{\partial \mathbf{s}}$ 。

3. OTTT 的关键近似

OTTT 观察到： $\frac{\partial \mathbf{s}^{l+1}[i]}{\partial \mathbf{u}^{l+1}[i]}$ 是 Heaviside 函数的导数，几乎处处为 0。因此，在时间依赖连乘中忽略这一项是合理的：

$$\frac{\partial \mathbf{u}^{l+1}[i+1]}{\partial \mathbf{s}^{l+1}[i]} \frac{\partial \mathbf{s}^{l+1}[i]}{\partial \mathbf{u}^{l+1}[i]} \approx 0.$$

这一近似与许多 BPTT with SG 实现中 “detach reset” 的做法一致。

于是连乘简化为：

$$\prod_{i=\tau}^{t-1} \frac{\partial \mathbf{u}^{l+1}[i+1]}{\partial \mathbf{u}^{l+1}[i]} = \lambda^{t-\tau} \mathbf{I}.$$

这里 λ 是 LIF 神经元的泄漏常数（通常取 0.5）， $\mathbf{I}$ 是单位矩阵。其物理含义：膜电位每步按 λ 衰减， $t - \tau$ 步后衰减为 $\lambda^{t-\tau}$ 。

4. OTTT 梯度公式的推导

代入简化后的连乘：

$$\frac{\partial L}{\partial \mathbf{W}^l} = \sum_{t=1}^T \mathbf{g}_{\mathbf{u}^{l+1}}[t] \left(\sum_{\tau \leq t} \lambda^{t-\tau} \mathbf{s}^l[\tau] \right)^\top. \quad (4)$$

其中 $\mathbf{g}_{\mathbf{u}^{l+1}}[t] = \left(\frac{\partial L}{\partial \mathbf{s}^{l+1}[t]} \frac{\partial \mathbf{s}^{l+1}[t]}{\partial \mathbf{u}^{l+1}[t]} \right)^\top$ 是 t 时刻的空间梯度。

5. 迹的递归定义

定义 $\hat{\mathbf{a}}^l[t] = \sum_{\tau \leq t} \lambda^{t-\tau} \mathbf{s}^l[\tau]$ 。这是一个指数加权的历史脉冲和。其递归更新：

$$\hat{\mathbf{a}}^l[t+1] = \lambda \hat{\mathbf{a}}^l[t] + \mathbf{s}^l[t+1]. \quad (4)$$

因此每步的梯度可以独立计算：

$$\nabla_{\mathbf{W}^l} L[t] = \mathbf{g}_{\mathbf{u}^{l+1}}[t] \cdot (\hat{\mathbf{a}}^l[t])^\top. \quad (4)$$

6. 内存复杂度分析

OTTT 需要存储的额外状态：

- $\hat{\mathbf{a}}^l[t]$ ：每个神经元一个标量，共 $\mathcal{O}(n)$ 。
- 不需要存储历史脉冲序列。
- 不需要存储 $\frac{\partial \mathbf{u}}{\partial \mathbf{u}}$ 矩阵。

总内存： $\mathcal{O}(n)$ ，与时间步长 T 无关。

7. 理论保证

定理 1 的核心结论是 OTTT 梯度是有效的下降方向：

$$\langle \nabla_{\text{OTTT}} L, \nabla_{\text{SR}} L \rangle > 0.$$

其中 $\nabla_{\text{SR}} L$ 是脉冲表示法（spike representation）的梯度，该方法已被证明提供下降方向。证明的直观思路：OTTT 梯度与 $\nabla_{\text{SR}} L$ 的差异被控制在一定范围内，使得两者的内积为正。

V. REPRESENT WORK—NDOT (NEURONAL DYNAMICS-BASED ONLINE TRAINING)

NDOT: Neuronal Dynamics-based Online Training (2024)

OTTT's Assumption Questioned:

- OTTT uses fixed decay λ .
- NDOT: *decay should depend on the neuron's actual state.*

The Continuous-Time Insight:

- From LIF dynamics: $e(t) = u'(t+1)/u'(t)$.
- Discrete version: $e^l[t] = (u^l[t] - V_{th}s^l[t])/(u^l[t-1] - V_{th}s^l[t-1])$.

Core Update:

$$\hat{a}^{l-1}[t] = e^{l-1}[t-1] \odot \hat{a}^{l-1}[t-1] + s^{l-1}[t] \quad (5)$$

Dynamic e replaces fixed λ . Same $\mathcal{O}(n)$ memory, higher accuracy.

【学生版讲稿】

NDOT 是 Jiang 等人于 2024 年提出的算法，它指出了 OTTT 的一个基本假设的局限：OTTT 使用固定的衰减因子 λ （通常取 0.5），但真实神经元的衰减应该取决于膜电位的实际状态。NDOT 从连续 LIF 动力学出发，推导出了精确的动态衰减因子 $e(t) = u'(t+1)/u'(t)$ 。离散化后得到 $e^l[t] = (u^l[t] - V_{th}s^l[t])/(u^l[t-1] - V_{th}s^l[t-1])$ 。这个动态 e 反映了神经元膜电位的“存活比例”——如果膜电位高（接近阈值），记忆保留更多；如果膜电位低（处于静息），记忆衰减更快。用这个动态 e 替代 OTTT 的固定 λ ，NDOT 在保持 $\mathcal{O}(n)$ 内存的同时，在 CIFAR-100 上将准确率从 OTTT 的 71.11% 提升到了 76.61%。

【自我学习笔记】

1. 算法定位与动机

NDOT (Jiang et al., 2024) 指出，OTTT 使用固定 λ 虽然简化了计算，但丢失了神经元状态的关键信息。在真实的生物神经元中，时间常数不是恒定的——它取决于膜电位相对于阈值的位置。NDOT 从连续时间 LIF 模型出发，推导出了精确的动态衰减因子。

2. 连续 LIF 动力学的完整推导

连续 LIF 方程：

$$\tau \frac{du}{dt} = -u(t) + I(t). \quad (1)$$

（忽略 u_{rest} ，不影响推导。）

定义连续时间资格迹：

$$e(t) \triangleq \frac{\partial u(t+1)}{\partial u(t)} = \frac{u'(t+1)}{u'(t)}. \quad (7)$$

由 LIF 方程， $u'(t) = [-u(t) + I(t)]/\tau$ ，因此：

$$e(t) = \frac{-u(t+1) + I(t+1)}{-u(t) + I(t)} = \frac{u(t+1) - I(t+1)}{u(t) - I(t)}. \quad (8)$$

离散化（步长 Δt ， $\lambda = e^{-\Delta t/\tau}$ ），带复位机制：

$$u[t+1] = \lambda(u[t] - V_{th}s[t]) + I[t+1].$$

代入式 (8)：

$$e^l[t] = \frac{\lambda(u^l[t] - V_{th}s^l[t]) + I^l[t+1] - I^l[t+1]}{u^l[t-1] - V_{th}s^l[t-1]} = \frac{u^l[t] - V_{th}s^l[t]}{u^l[t-1] - V_{th}s^l[t-1]} \cdot \lambda.$$

论文中略去 λ 因子（可吸收到阈值 V_{th} ），得：

$$e^l[t] = \frac{u^l[t] - V_{th}s^l[t]}{u^l[t-1] - V_{th}s^l[t-1]}. \quad (8)$$

3. 动态 e 的物理解释

$e^l[t]$ 的分子 $u^l[t] - V_{th}s^l[t]$ 是神经元放电后的膜电位（若 $s = 1$ 则减去阈值，否则不变）。分母 $u^l[t-1] - V_{th}s^l[t-1]$ 是上一时刻放电后的膜电位。因此 e 衡量了膜电位从上一时刻到当前时刻的“存活比例”：

- 若 $u^l[t]$ 接近阈值， e 较大，记忆保留更多。
- 若 $u^l[t]$ 远离阈值（静息）， e 较小，记忆衰减更快。
- 放电后 $s = 1$ 使 $u^l[t]$ 减去 V_{th} ， e 急剧减小，对应不应期。

这与生物神经元的特性完全一致。

4. NDOT 的时间梯度更新

用动态 e 替代固定 λ ：

$$\hat{a}^{l-1}[t] = e^{l-1}[t-1] \odot \hat{a}^{l-1}[t-1] + s^{l-1}[t]. \quad (12)$$

其中 $\odot$ 是逐元素乘法。 $e^{l-1}[t-1]$ 的每个分量对应一个神经元的独立衰减因子。

VI. BPTT: FULL DERIVATION AND PARAMETER ORIGINS / BPTT 详细推导

BPTT: Full Derivation and Parameter Origins

Step 1 — Forward dynamics:

$$u_j^l[t+1] = \lambda(u_j^l[t] - V_{th}s_j^l[t]) + \sum_i W_{ji}^l s_i^{l-1}[t+1] + b_j^l \quad (6)$$

$$s_j^l[t] = H(u_j^l[t] - V_{th}) \quad (7)$$

Step 2 — Chain rule decomposition (with total derivative):

$$\frac{dL}{dW_{ji}^l} = \sum_t \frac{\partial L}{\partial s_j^l[t]} \cdot \frac{\partial s_j^l[t]}{\partial u_j^l[t]} \cdot \frac{du_j^l[t]}{dW_{ji}^l} \quad (8)$$

Note: the last factor is a total derivative, not a partial derivative.

Step 3 — Surrogate for spike derivative:

$$\frac{\partial s_j^l[t]}{\partial u_j^l[t]} \rightarrow \Psi'(u_j^l[t] - V_{th}), \quad D_{\Psi,t} = \text{diag}[\Psi'(u_j^l[t] - V_{th})] \quad (9)$$

Step 4 — Expand the total derivative of membrane potential:

$$\frac{du_j^l[t]}{dW_{ji}^l} = \sum_{\tau \leq t} \frac{\partial u_j^l[t]}{\partial u_j^l[\tau]} \cdot \frac{\partial u_j^l[\tau]}{\partial W_{ji}^l} \quad (10)$$

Step 5 — Define the temporal dependency matrix:

$$\epsilon^l[i] := \frac{\partial u^l[i+1]}{\partial u^l[i]} \quad (11)$$

Step 6 — Derive $\epsilon^l[i]$ from LIF:

$$\epsilon^l[i] = \underbrace{\lambda I}_{\text{leak}} - \underbrace{\lambda V_{th} D_{\Psi,i}}_{\text{reset}} \quad (12)$$

Step 7 — Ordered temporal product:

$$\mathcal{E}_{t,\tau} = \epsilon^l[t-1]\epsilon^l[t-2]\cdots\epsilon^l[\tau] \quad (13)$$

Step 8 — Final BPTT gradient:

$$\frac{dL}{dW^{l \leftarrow l-1}} = \sum_t \frac{\partial L}{\partial s^l[t]} \frac{\partial s^l[t]}{\partial u^l[t]} \left(\frac{\partial u^l[t]}{\partial W^{l \leftarrow l-1}} + \sum_{\tau < t} \mathcal{E}_{t,\tau} \frac{\partial u^l[\tau]}{\partial W^{l \leftarrow l-1}} \right) \quad (14)$$

Note: bracket equals $du^l[t]/dW^{l \leftarrow l-1}$; inner partials are direct dependencies.

Memory cost: $O(TLn)$ — must store all forward states.

【公式详细每一步推导】

Step 1 — 前向动力学

LIF 神经元的离散更新:

$$u_j^l[t+1] = \lambda(u_j^l[t] - V_{th}s_j^l[t]) + \sum_i W_{ji}^l s_i^{l-1}[t+1] + b_j^l, \quad (15)$$

$$s_j^l[t] = H(u_j^l[t] - V_{th}). \quad (16)$$

参数来源与服务作用:

- $\lambda = e^{-\delta t/\tau_m} \in (0, 1)$: 来自连续 LIF 方程 $\tau_m \dot{u} = -u + I$ 的离散化; δt 是离散步长, τ_m 是膜时间常数。作用: 膜电位随时间的指数衰减率。
- V_{th} : 来自 LIF 模型定义。作用: 发放阈值, 同时作为 reset 幅度。
- W_{ji}^l : 可学习参数。作用: 将突触前脉冲 s_i^{l-1} 加权为输入电流。
- b_j^l : 可学习偏置。作用: 调整神经元兴奋基线。
- $H(x) = \mathbb{I}[x \geq 0]$: Heaviside 阶跃函数。作用: 产生二值脉冲。

Step 2 — 链式法则分解 (含全导)

目标: $\frac{dL}{dW_{ji}^l}$ 。链式法则给出:

$$\frac{dL}{dW_{ji}^l} = \sum_t \frac{\partial L}{\partial s_j^l[t]} \cdot \frac{\partial s_j^l[t]}{\partial u_j^l[t]} \cdot \frac{du_j^l[t]}{dW_{ji}^l}. \quad (17)$$

关键: 最后一个因子是全导, 不是偏导。

W_{ji}^l 通过两条路径影响 $u_j^l[t]$:

- 直接路径: t 时刻的输入 $W_{ji}^l s_i^{l-1}[t]$ 进入 $u_j^l[t]$;

- **间接路径:** W_{ji}^l 影响 $u_j^l[t-1]$, 再经过状态转移到 $u_j^l[t]$ 。

三个因子各自的来源与含义:

- $\partial L / \partial s_j^l[t]$: 来自损失函数 L 对脉冲 $s_j^l[t]$ 的直接依赖。对输出层是损失函数的直接导数; 对隐藏层来自下一层反向传播。
- $\partial s_j^l[t] / \partial u_j^l[t]$: 脉冲生成函数的导数。来自 $s_j^l[t] = H(u_j^l[t] - V_{th})$, 用代理梯度替换。
- $du_j^l[t] / dW_{ji}^l$: 膜电位对权重的全导数。展开见 Step 4。

Step 3 — 代理梯度替换

$s_j^l[t] = H(u_j^l[t] - V_{th})$ 几乎处处导数为零, 不可直接用于反向传播。用一个光滑函数替代:

$$\frac{\partial s_j^l[t]}{\partial u_j^l[t]} \rightarrow \Psi'(u_j^l[t] - V_{th}). \quad (18)$$

本文采用 fast-sigmoid: $\Psi'(x) = 1/(1 + \beta|x|)^2$, $\beta = 4$ 。记:

$$D_{\Psi,t} = \text{diag}[\Psi'(u_j^l[t] - V_{th})]_j. \quad (19)$$

参数来源与作用:

- Ψ' : 代理梯度函数, 是本文的实验选择, 不是 LIF 模型本身的性质。
- β : 锐度参数, 控制代理梯度的衰减速度。
- $D_{\Psi,t}$: 代理梯度对角矩阵, 用于后续的 $\epsilon^l[i]$ 构造。

Step 4 — 展开膜电位的全导数

$du_j^l[t] / dW_{ji}^l$ 是全导, 因为 $u_j^l[t]$ 通过历史时刻 $u_j^l[\tau]$ ($\tau < t$) 间接依赖 W_{ji}^l :

$$\frac{du_j^l[t]}{dW_{ji}^l} = \sum_{\tau \leq t} \frac{\partial u_j^l[t]}{\partial u_j^l[\tau]} \cdot \frac{\partial u_j^l[\tau]}{\partial W_{ji}^l}. \quad (20)$$

其中:

- $\partial u_j^l[t] / \partial u_j^l[\tau]$: 从 τ 到 t 的状态转移链;
- $\partial u_j^l[\tau] / \partial W_{ji}^l$: τ 时刻的直接注入, 即 $s_i^{l-1}[\tau]$ 。

注意: 展开后的 $\partial u_j^l[\tau] / \partial W_{ji}^l$ 是偏导, 因为 $u_j^l[\tau]$ 对 W_{ji}^l 的直接依赖只通过 τ 时刻那一项。

Step 5 — 定义时间依赖矩阵

将状态转移链写成 Jacobian 连乘, 引入时间依赖矩阵:

$$\epsilon^l[i] := \frac{\partial u^l[i+1]}{\partial u^l[i]}. \quad (21)$$

那么从 τ 到 t 的传播为 $\mathcal{E}_{t,\tau} = \epsilon^l[t-1]\epsilon^l[t-2]\cdots\epsilon^l[\tau]$ 。

Step 6 — 从 LIF 推导 $\epsilon^l[i]$ 的两项

从 LIF 更新出发:

$$u^l[i+1] = \lambda u^l[i] - \lambda V_{th} s^l[i] + W^l s^{l-1}[i+1] + b^l. \quad (22)$$

路径 1 (泄漏): 对 $u^l[i]$ 求 $\lambda u^l[i]$ 的偏导:

$$\frac{\partial(\lambda u^l[i])}{\partial u^l[i]} = \lambda I. \quad (23)$$

路径 2 (复位): $s^l[i] = H(u^l[i] - V_{th})$, 通过代理梯度:

$$\frac{\partial(-\lambda V_{th} s^l[i])}{\partial u^l[i]} = -\lambda V_{th} \cdot D_{\Psi,i}. \quad (24)$$

相加:

$$\epsilon^l[i] = \lambda I - \lambda V_{th} D_{\Psi,i}. \quad (25)$$

参数来源与作用:

- λI : 来自膜泄漏项。表示膜电位本身按 λ 衰减。
- $-\lambda V_{th} D_{\Psi,i}$: 来自复位项。表示脉冲发放后膜电位被减去 V_{th} , 其大小为 V_{th} , 并受代理梯度 $D_{\Psi,i}$ 调节。

Step 7 — 有序时间连乘

$$\mathcal{E}_{t,\tau} = \epsilon^l[t-1] \epsilon^l[t-2] \cdots \epsilon^l[\tau], \quad \tau < t. \quad (26)$$

矩阵乘法不可交换, 因此顺序必须从 $t-1$ 递减到 τ 。边界约定: $\mathcal{E}_{t,t} = I$ (空乘积)。

Step 8 — BPTT 主公式

将 Step 4、5、7 代回 Step 2:

$$\frac{dL}{dW^{l \leftarrow l-1}} = \sum_{t=1}^T \frac{\partial L}{\partial s^l[t]} \frac{\partial s^l[t]}{\partial u^l[t]} \left(\frac{\partial u^l[t]}{\partial W^{l \leftarrow l-1}} + \sum_{\tau < t} \mathcal{E}_{t,\tau} \frac{\partial u^l[\tau]}{\partial W^{l \leftarrow l-1}} \right). \quad (27)$$

括号里的表达式就是全导 $du^l[t]/dW^{l \leftarrow l-1}$ 。里面两项是偏导, 因为它们分别只捕捉直接影响 ($\tau = t$) 和间接影响 ($\tau < t$)。

公式解读:

- 第一项 (直接影响): 当前时刻 t , 权重通过输入 $s^{l-1}[t]$ 直接进入 $u^l[t]$, 即 $s^{l-1}[t]^\top$ 。
- 第二项 (历史路径): 权重在时刻 τ 注入的输入, 经过 $\mathcal{E}_{t,\tau}$ 传播到当前。
- 外面的 $\partial L/\partial s^l[t] \cdot \partial s^l[t]/\partial u^l[t]$: 把膜电位对损失的贡献转换为误差信号。

内存分析

BPTT 反向时需要 $\epsilon^l[i]$, 而 $\epsilon^l[i]$ 依赖 $u^l[i]$ 和 $s^l[i]$ 。因此必须存储所有时间步的 forward states:

$$\{u^l[t], s^l[t]\}_{t=1, t=1}^{L,T}. \quad (28)$$

总内存:

$$O(T \cdot L \cdot n). \quad (29)$$

这不是实现细节, 而是 BPTT 算法本身的代价。

参数小结表

参数	来源	作用
λ	连续 LIF 离散化 $e^{-\delta t/\tau_m}$	膜电位指数衰减
V_{th}	LIF 模型定义	发放阈值 + reset 幅度
W_{ji}^l, b_j^l	可学习参数	突触权重、偏置
$H(\cdot)$	脉冲生成	二值脉冲
Ψ'	代理梯度选择	替代不可导的 $\partial s/\partial u$
$D_{\Psi,t}$	$\text{diag}[\Psi'(u - V_{th})]$	代理导数对角矩阵
$\epsilon^l[i]$	LIF Jacobian	时间依赖矩阵
$\mathcal{E}_{t,\tau}$	有序连乘	从 τ 到 t 的时间传播

VII. OTTT: FROM BPTT TO FIXED DECAY — FULL DERIVATION / OTTT 详细推导

OTTT: From BPTT to Fixed Decay

Step 1 — Start from BPTT's temporal chain:

$$\epsilon^l[i] = \underbrace{\lambda I}_{\text{leak}} - \underbrace{\lambda V_{th} D_{\Psi,i}}_{\text{reset via spike}} \quad (30)$$

Step 2 — Observe $D_{\Psi,i}$ is near zero almost everywhere:

$$\frac{\partial u^{l+1}[i+1]}{\partial s^{l+1}[i]} \frac{\partial s^{l+1}[i]}{\partial u^{l+1}[i]} \approx 0 \quad (31)$$

Step 3 — Drop the reset term:

$$\epsilon^l[i] \approx \lambda I \quad (32)$$

Step 4 — Temporal product collapses to exponential decay:

$$\mathcal{E}_{t,\tau} = \prod_{i=\tau}^{t-1} \epsilon^l[i] \approx \lambda^{t-\tau} I \quad (33)$$

Step 5 — Substitute into BPTT gradient (using本文 convention $W^{l \leftarrow l-1}$):

$$\frac{dL}{dW^{l \leftarrow l-1}} = \sum_t g_{u^l}[t] \left(\sum_{\tau \leq t} \lambda^{t-\tau} s^{l-1}[\tau] \right)^\top \quad (34)$$

$$g_{u^l}[t] = \left(\frac{\partial L}{\partial s^l[t]} \frac{\partial s^l[t]}{\partial u^l[t]} \right)^\top \quad (35)$$

Step 6 — Define the presynaptic trace:

$$\hat{a}^{l-1}[t] := \sum_{\tau \leq t} \lambda^{t-\tau} s^{l-1}[\tau] \quad (36)$$

Step 7 — Recursive update:

$$\hat{a}^{l-1}[t+1] = \lambda \hat{a}^{l-1}[t] + s^{l-1}[t+1] \quad (37)$$

Step 8 — OTTT instantaneous gradient:

$$\nabla_{W^{l \leftarrow l-1}} L[t] = g_{u^l}[t] \cdot \hat{a}^{l-1}[t]^\top \quad (38)$$

Memory: $O(Ln)$, independent of T .

【公式详细每一步推导】

Step 1 — BPTT 时间连乘的两项

从 BPTT 的时间依赖矩阵出发：

$$\epsilon^l[i] = \underbrace{\lambda I}_{\text{路径 1: 泄漏}} \underbrace{-\lambda V_{th} D_{\Psi,i}}_{\text{路径 2: 复位}}. \quad (39)$$

两项的来源：

- λI ：来自 LIF 中的 $\lambda u^l[i]$ 项。对 $u^l[i]$ 求偏导得 λI 。
- $-\lambda V_{th} D_{\Psi,i}$ ：来自 LIF 中的 $-\lambda V_{th} s^l[i]$ 项。因为 $s^l[i] = H(u^l[i] - V_{th})$ ，对 $u^l[i]$ 求偏导得 $D_{\Psi,i}$ ，所以这一项对 $u^l[i]$ 的偏导是 $-\lambda V_{th} D_{\Psi,i}$ 。

Step 2 — 复位项的数值量级

代理梯度 $D_{\Psi,i}$ 的第 j 个对角元是 $\Psi'(u_j^l[i] - V_{th})$ 。对于 fast-sigmoid $\Psi'(x) = 1/(1 + \beta|x|)^2$ ：

- $x = 0$ （膜电位等于阈值）： $\Psi' = 1$ ，最大。
- $|x| = 1$ ： $\Psi' = 1/25 = 0.04$ 。
- $|x| = 10$ ： $\Psi' = 1/1681 \approx 6 \times 10^{-4}$ 。

真实网络中大多数时间步的膜电位远离阈值，因此 $D_{\Psi,i} \approx 0$ 。即使在阈值附近，由于连乘的存在，任何一步 $D_{\Psi,i}$ 接近零就会让整个路径大幅衰减。

Step 3 — 丢弃复位项

OTTT 直接丢弃复位项：

$$\frac{\partial u^{l+1}[i+1]}{\partial s^{l+1}[i]} \frac{\partial s^{l+1}[i]}{\partial u^{l+1}[i]} = -\lambda V_{th} D_{\Psi,i} \approx 0. \quad (40)$$

于是：

$$\boxed{\epsilon^l[i] \approx \lambda I}. \quad (41)$$

物理意义：只保留膜电位的连续指数衰减，忽略脉冲发放带来的离散复位效应。

Step 4 — 连乘退化为指数衰减

时间连乘：

$$\mathcal{E}_{t,\tau} = \prod_{i=\tau}^{t-1} \epsilon^l[i] \approx \prod_{i=\tau}^{t-1} (\lambda I) = \lambda^{t-\tau} I. \quad (42)$$

物理意义：从 τ 到 t 的传播只依赖时间差 $t - \tau$ ，不依赖中间任何时刻的状态。这就是“固定衰减”的来源。

Step 5 — 代入 BPTT 梯度

将 $\mathcal{E}_{t,\tau} \approx \lambda^{t-\tau} I$ 代入 BPTT 主公式:

$$\frac{dL}{dW^{l \leftarrow l-1}} = \sum_{t=1}^T g_{u^l}[t] \left(\sum_{\tau \leq t} \lambda^{t-\tau} s^{l-1}[\tau] \right)^\top, \quad (43)$$

其中

$$g_{u^l}[t] = \left(\frac{\partial L}{\partial s^l[t]} \frac{\partial s^l[t]}{\partial u^l[t]} \right)^\top. \quad (44)$$

参数来源与作用:

- $g_{u^l}[t]$: 当前时刻的空间梯度。通过当前时刻的 `autograd` 得到, 不跨时间反传。
- $\lambda^{t-\tau}$: 来自丢弃复位项后的连乘。表示从 τ 到 t 的指数衰减权重。
- $s^{l-1}[\tau]$: 突触前脉冲。来自上一层的前向状态。

Step 6 — 定义预突触迹

括号里的求和只依赖 t 和层 $l-1$, 定义为:

$$\hat{a}^{l-1}[t] := \sum_{\tau \leq t} \lambda^{t-\tau} s^{l-1}[\tau]. \quad (45)$$

物理意义: 到时刻 t 为止, 层 $l-1$ 所有历史脉冲的指数加权和。越近的脉冲权重越大 ($\lambda^0 = 1$), 越远的越小 (λ^k)。

Step 7 — 递推更新

推导递推关系:

$$\hat{a}^{l-1}[t+1] = \sum_{\tau \leq t+1} \lambda^{t+1-\tau} s^{l-1}[\tau] \quad (46)$$

$$= s^{l-1}[t+1] + \lambda \sum_{\tau \leq t} \lambda^{t-\tau} s^{l-1}[\tau] \quad (47)$$

$$= s^{l-1}[t+1] + \lambda \hat{a}^{l-1}[t]. \quad (48)$$

即:

$$\hat{a}^{l-1}[t+1] = \lambda \hat{a}^{l-1}[t] + s^{l-1}[t+1]. \quad (49)$$

关键意义: 只需要维护一个向量 $\hat{a}^{l-1}[t] \in \mathbb{R}^{n^{(l-1)}}$, 不需要保存历史脉冲。这就是内存从 $O(TLn)$ 降到 $O(Ln)$ 的机制。

Step 8 — OTTT 的瞬时梯度

$$\nabla_{W^{l \leftarrow l-1}} L[t] = g_{u^l}[t] \cdot \hat{a}^{l-1}[t]^\top. \quad (50)$$

参数总结:

- $g_{u^l}[t]$: 当前时刻空间梯度。
- $\hat{a}^{l-1}[t]$: 历史 `presynaptic spikes` 的指数加权迹。
- λ : 膜衰减因子, 同时成为时间迹的衰减系数。

- V_{th} : 只影响空间梯度（通过代理梯度），不进入迹递推。

内存分析

需要维护的状态:

- 每层一个 $\hat{a}^{l-1}[t] \in \mathbb{R}^{n^{(l-1)}}$: 共 $O(Ln)$ 。
- 当前时刻的 $u^l[t], s^l[t]$: 共 $O(Ln)$ 。

不需要维护: 历史脉冲序列、历史 Jacobian、任何 $O(T)$ 或 $O(n^2)$ 对象。

总内存 $O(Ln)$, 与 T 无关。

两种更新模式

- **OTTT_O (Online)**: 每步用 $\nabla_{W^{l \leftarrow l-1}} L[t]$ 立即更新。
- **OTTT_A (Accumulated)**: 累计 $\sum_t \nabla_{W^{l \leftarrow l-1}} L[t]$ 后统一更新。

梯度保真度比较应使用 OTTT_A, 因为 OTTT_O 在时间步之间改变参数, 导致不同时间步的梯度不在同一评估点。

参数小结表

参数	来源	作用
λ	膜衰减因子	时间迹的指数衰减
$\hat{a}^{l-1}[t]$	递推定义	历史 presynaptic 活动
$g_{u^l}[t]$	当前时刻 autograd	空间梯度
V_{th}	LIF 阈值	只影响空间梯度, 不进入迹递推
T	序列长度	决定总更新步数

VIII. NDOT: DYNAMIC DECAY — FULL DERIVATION / NDOT 详细推导

NDOT: Dynamic Decay from Neuron State

Step 1 — Question OTTT's fixed λ :

$$\text{OTTT: } \epsilon^l[i] \approx \lambda I, \quad \lambda \text{ same for all neurons, all } t \quad (51)$$

Step 2 — Continuous LIF dynamics:

$$\tau \frac{du}{dt} = -u(t) + I(t) \quad (52)$$

Step 3 — Define continuous dynamic coefficient:

$$e(t) := \frac{\partial u(t+1)}{\partial u(t)} = \frac{u'(t+1)}{u'(t)} \quad (53)$$

Step 4 — Substitute LIF into $e(t)$:

$$e(t) = \frac{-u(t+1) + I(t+1)}{-u(t) + I(t)} = \frac{u(t+1) - I(t+1)}{u(t) - I(t)} \quad (54)$$

Step 5 — Discretize:

$$u^l[t+1] - I^l[t+1] = \lambda(u^l[t] - V_{th}s^l[t]) \quad (55)$$

$$e^l[t] = \frac{u^l[t] - V_{th}s^l[t]}{u^l[t-1] - V_{th}s^l[t-1]} \quad (56)$$

Step 6 — Boundary case at $t = 1$:

$$e^l[1] = u^l[1] - V_{th}s^l[1] \quad (57)$$

Step 7 — NDOT temporal trace:

$$\hat{a}^{l-1}[t] = e^{l-1}[t-1] \odot \hat{a}^{l-1}[t-1] + s^{l-1}[t] \quad (58)$$

Step 8 — NDOT instantaneous gradient:

$$\nabla_{W^{l \leftarrow l-1}} L[t] = g_{u^l}[t] \cdot \hat{a}^{l-1}[t]^\top \quad (59)$$

Same memory as OTTT ($O(Ln)$), better accuracy.

【公式详细每一步推导】

Step 1 — OTTT 固定 λ 的局限

OTTT 用 $\epsilon^l[i] \approx \lambda I$ ，即所有神经元所有时刻用同一个 λ 。但在真实网络中：

- 神经元处于活跃状态（ u 接近 V_{th} ）：实际衰减比 λ 慢。
- 神经元刚发放完（ u 被 reset 到低值）：实际衰减比 λ 快。
- 不同神经元状态差异大： λ 无法反映这种异质性。

NDOT 的目标：用依赖神经元当前状态的动态系数替代固定 λ 。

Step 2 — 连续 LIF 动力学

连续 LIF 方程：

$$\tau \frac{du}{dt} = -u(t) + I(t), \quad (60)$$

其中 $I(t)$ 是输入电流， τ 是膜时间常数。

参数来源与作用：

- $u(t)$ ：膜电位，来自 LIF 模型。
- $I(t)$ ：输入电流，来自突触前活动和外部输入。
- τ ：膜时间常数，来自生物神经元特性。

Step 3 — 定义动态系数

定义连续时间动态系数：

$$e(t) := \frac{\partial u(t+1)}{\partial u(t)} = \frac{u'(t+1)}{u'(t)}. \quad (61)$$

为什么用 $u'(t+1)/u'(t)$ ：这是 $u(t+1)$ 对 $u(t)$ 的导数。在连续时间下，它反映“ u 从 t 到 $t+1$ 的变化率之比”。

Step 4 — 代入 LIF

由 LIF 方程 $u'(t) = (-u(t) + I(t))/\tau$ 。代入：

$$e(t) = \frac{u'(t+1)}{u'(t)} = \frac{-u(t+1) + I(t+1)}{-u(t) + I(t)} = \frac{u(t+1) - I(t+1)}{u(t) - I(t)}. \quad (62)$$

τ 在分子分母中约掉。

Step 5 — 离散化

离散 LIF:

$$u^l[t+1] = \lambda(u^l[t] - V_{th}s^l[t]) + I^l[t+1], \quad (63)$$

其中 $I^l[t+1]$ 是外部输入电流。把 $u^l[t+1]$ 移项:

$$u^l[t+1] - I^l[t+1] = \lambda(u^l[t] - V_{th}s^l[t]). \quad (64)$$

同理 $u^l[t] - I^l[t] = \lambda(u^l[t-1] - V_{th}s^l[t-1])$ 。代入 $e(t)$ 分子分母:

$$e^l[t] = \frac{\lambda(u^l[t] - V_{th}s^l[t])}{\lambda(u^l[t-1] - V_{th}s^l[t-1])} = \frac{u^l[t] - V_{th}s^l[t]}{u^l[t-1] - V_{th}s^l[t-1]}. \quad (65)$$

λ 约掉。

Step 6 — 边界情况 $t = 1$

$t = 1$ 时分母 $u^l[0] - V_{th}s^l[0]$ 涉及初始状态，可能为零或未定义。NDOT 规定:

$$e^l[1] = u^l[1] - V_{th}s^l[1]. \quad (66)$$

即 $t = 1$ 时取放电后的膜电位，避免除零。

Step 7 — NDOT 时间迹递推

用 $e^{l-1}[t-1]$ 替代 OTTT 中的 λ :

$$\hat{a}^{l-1}[t] = e^{l-1}[t-1] \odot \hat{a}^{l-1}[t-1] + s^{l-1}[t]. \quad (67)$$

关键区别: OTTT 用标量 λ 乘整个 $\hat{a}$; NDOT 用向量 $e^{l-1}[t-1]$ 逐元素乘。

展开为历史求和:

$$\hat{a}^{l-1}[t] = s^{l-1}[t] + \sum_{k < t} (e^{l-1}[t-1] \odot e^{l-1}[t-2] \odot \cdots \odot e^{l-1}[k] \odot s^{l-1}[k]). \quad (68)$$

每一项都是逐元素的，因此不同神经元的历史衰减路径不同。

Step 8 — NDOT 瞬时梯度

$$\nabla_{W^{l \leftarrow l-1}} L[t] = g_{u^l}[t] \cdot \hat{a}^{l-1}[t]^\top. \quad (69)$$

参数来源与作用:

- $e^l[t]$: 逐神经元动态衰减因子。来自放电后膜电位比值。
- $\hat{a}^{l-1}[t]$: 动态衰减的时间迹。
- $g_{u^l}[t]$: 当前时刻空间梯度。
- V_{th} : reset 幅度 + 分母偏移。

数值稳定处理

NDOT 论文 Appendix A.2 规定: 当 $e^l[t]$ 的分母 $u^l[t-1] - V_{th}s^l[t-1]$ 接近零时:

- 根据膜电位符号处理退化情况。
- 用 clamping 把结果限制在 $[-\lambda, \lambda]$ 。

这个处理只对分母为零的退化情况生效，不是一般定义。

内存与计算

内存：需要维护 $\hat{a}^{l-1}[t] \in \mathbb{R}^{n^{(l-1)}}$ 每层一个。加上当前时刻 u^l, s^l 。总内存 $O(Ln)$ 。

计算：每步计算 $e^{l-1}[t-1]$ 需要逐元素除法，额外开销约 $O(Ln)$ 。

与 OTTT 的对比

方面	OTTT	NDOT
时间系数	固定 λ	动态 $e^l[t]$
迹递推	$\hat{a}[t+1] = \lambda\hat{a}[t] + s[t+1]$	$\hat{a}[t] = e[t-1] \odot \hat{a}[t-1] + s[t]$
内存	$O(Ln)$	$O(Ln)$
额外计算	无	逐元素除法
时间依赖刻画	全局统一	神经元独立

参数小结表

参数	来源	作用
$e^l[t]$	相邻时刻放电后膜电位比值	逐神经元动态衰减
$u^l[t]$	前向动力学	提供动态系数
$s^l[t]$	脉冲生成	决定是否减去 V_{th}
$\hat{a}^{l-1}[t]$	递推	动态衰减时间迹
$g_{u^l}[t]$	当前 autograd	空间梯度
V_{th}	LIF 阈值	reset 幅度 + 分母偏移
α	超参	CE 与 MSE 权重

IX. REPRESENT WORK—S-TLLR: STDP-INSPIRED TEMPORAL LOCAL LEARNING RULE

S-TLLR: STDP-inspired Temporal Local Learning (2025)

Beyond Causality:

- BPTT only uses causal relations (pre $\rightarrow$ post).
- STDP can also reward synchrony (post $\rightarrow$ pre).
- S-TLLR: *learn from both*.

The Non-Causal Term:

- Parameter α_{post} controls non-causal reward.
- $\alpha_{post} > 0$: reward synchrony; $\alpha_{post} < 0$: punish it.
- Vision prefers causality (-1); audio prefers synchrony ($+1$).

Key Formula:

$$e_{ij}[t] = \alpha_{pre} \Psi(u_i[t]) \text{tr}(x_j)[t] + \alpha_{post} x_j[t] (\text{tr}(\Psi(u_i[t])) - \Psi(u_i[t])) \quad (70)$$

Memory $\mathcal{O}(n)$ — no extra cost for extra information.

【学生版讲稿】

S-TLLR 是 Apolinario 和 Roy 于 2025 年发表在 TMLR 上的工作，它挑战了 BPTT 及其所有近似的隐含假设：只考虑因果关系（突触前神经元先发放，突触后神经元后发放）。然而，STDP 的生物学实验（Anisimova et al., 2022）表明，长期演化的 STDP 更倾向于奖励同步性而非纯粹的因果性。S-TLLR 将这一观察参数化为一个可调节的非因果项，用 α_{post} 控制其强度。当 $\alpha_{post} < 0$ 时，非因果项为负（传统 STDP）；当 $\alpha_{post} > 0$ 时，非因果项为正（奖励同步）。实验发现，视觉任务（DVS Gesture、CIFAR10-DVS）偏好 $\alpha_{post} = -1$ （强调因果），而音频任务（SHD）偏好 $\alpha_{post} = +1$ （强调同步）。S-TLLR 的资格迹包含两项：因果项 $\alpha_{pre}\Psi(u_i[t])\text{tr}(x_j)[t]$ 和非因果项 $\alpha_{post}x_j[t](\text{tr}(\Psi(u_i[t])) - \Psi(u_i[t]))$ 。内存仍为 $\mathcal{O}(n)$ ——获得更多信息不增加成本。

【自我学习笔记】

1. 算法定位与动机

S-TLLR (Apolinario & Roy, 2025, TMLR) 是第一个将 STDP 的非因果性系统引入在线学习的 SNN 训练算法。它的动机来自两个观察：

- 1) BPTT 及其所有近似只使用因果关系 ($x[0..t] \rightarrow y[t]$)。
- 2) STDP 的生物实验表明，长期来看，STDP 更倾向于奖励同步性而非因果性。

S-TLLR 问：我们能否从两者都学习？

2. 原始 STDP 方程

传统 STDP 描述：

$$\Delta w_{ij} = \begin{cases} A_+ e^{-(t_i - t_j)/\tau_+}, & t_i \geq t_j \quad (\text{LTP, 前} \rightarrow \text{后}) \\ -A_- e^{-(t_j - t_i)/\tau_-}, & t_i < t_j \quad (\text{LTD, 后} \rightarrow \text{前}) \end{cases}$$

其中 t_i 是突触后脉冲时间， t_j 是突触前脉冲时间。

3. S-TLLR 的广义参数化

$$\Phi(t_i, t_j) = \begin{cases} \alpha_{pre} \lambda_{pre}^{t_i - t_j}, & t_i \geq t_j \quad (\text{因果}) \\ \alpha_{post} \lambda_{post}^{t_j - t_i}, & t_i < t_j \quad (\text{非因果}) \end{cases} \quad (3)$$

参数含义：

- α_{pre} ：因果项强度（正值表示增强）。
- α_{post} ：非因果项强度。 $\alpha_{post} < 0$ ：惩罚同步； $\alpha_{post} > 0$ ：奖励同步。
- $\lambda_{pre}, \lambda_{post}$ ：衰减因子。

4. 基于迹的资格迹推导

用低通滤波轨迹代替精确脉冲时间：

$$\text{tr}(x_j)[t] = \sum_{t'=0}^t \lambda_{pre}^{t-t'} x_j[t'], \quad \text{tr}(\Psi(u_i))[t] = \sum_{t'=0}^{t-1} \lambda_{post}^{t-t'} \Psi(u_i[t']).$$

其中 Ψ 是二次激活函数（替代 Heaviside，提供平滑的导数）。

将 STDP 转换为迹形式：

$$e_{ij}[t] = \alpha_{pre} \Psi(u_i[t]) \text{tr}(x_j)[t] + \alpha_{post} x_j[t] (\text{tr}(\Psi(u_i[t])) - \Psi(u_i[t])). \quad (11)$$

每一项的物理含义：

- $\alpha_{pre} \Psi(u_i[t]) \text{tr}(x_j)[t]$ ：当前突触后活动乘以突触前迹——因果项。
- $\alpha_{post} x_j[t] (\text{tr}(\Psi(u_i[t])) - \Psi(u_i[t]))$ ：当前突触前活动乘以突触后迹（减去当前值避免自洽）——非因果项。

X. REPRESENT WORK—TESS: TEMPORALLY AND SPATIALLY LOCAL LEARNING RULE

TESS: Temporally & Spatially Local Learning (2025)

The Ultimate Goal: Full Locality:

- Previous: temporally local, but still need layer-to-layer error propagation.
- TESS: *can each layer learn from its own local signal?* Yes.

Two Keys:

- *Time locality*: instantaneous traces ($\beta = 0$) — memory $\mathcal{O}(n)$.
- *Space locality*: each layer projects outputs to label space and compares locally.

Core Formula:

$$m^l[t] = B^{l\top} (f(B^l o^l[t]) - y^*), \quad \Delta W^l[t] = m^l[t] \odot [\text{traces}] \quad (71)$$

The Result: No layer-to-layer propagation. Fully parallel. Performance matches BPTT. MACs reduced by $200\times$.

【学生版讲稿】

TESS 是 Apolinario 等人于 2025 年提出的算法，它实现了在线学习算法的终极目标：时间和空间的完全局部化。之前的算法虽然在时间上是局部的（通过 Eligibility Trace），但在空间上仍然需要跨层误差反向传播。TESS 突破了这一限制：每一层都可以用自己本地的信号学习。

TESS 的时间局部性来自瞬时资格迹（设 $\beta = 0$ ），内存为 $\mathcal{O}(n)$ 。空间局部性是其核心创新：每层用一个固定的准正交二进制矩阵 B^l 将本层输出投影到任务子空间（维度为类别数 C ），通过 softmax 与标签比较后，再投影回本层，生成本地学习信号 $m^l[t]$ 。这个过程完全不需要跨层传播误差。TESS 在 CIFAR-10 和 DVS Gesture 上达到了与 BPTT 相当的准确率，同时 MAC 操作减少了 200 倍以上。

【自我学习笔记】

1. 算法定位与动机

TESS (Apolinario et al., 2025) 是实现完全局部学习的第一个 SNN 训练算法。它结合了三种生物机制：Eligibility Trace（时间记忆）、STDP（时序依赖）和 Neural Activity Synchronization（空间投影）。目标是：每个神经元只用本地信号完成权重更新，无需任何全局信息。

2. 标准三点因子规则的内存瓶颈

标准三点因子规则：

$$e_{ij}^{(l)}[t] = \beta e_{ij}^{(l)}[t-1] + f(o_i^{(l)}[t])g(o_j^{(l-1)}[t]), \quad \Delta W_{ij} = \sum_t m_i[t]e_{ij}^{(l)}[t].$$

其中 $e_{ij}^{(l)}$ 是每个突触的资格迹，需要 $\mathcal{O}(n^2)$ 内存。TESS 设 $\beta = 0$ ，得到瞬时资格迹。

3. 时间局部性

突触前迹和后迹：

$$q^l[t] = \lambda_{pre} q^l[t-1] + o^{l-1}[t], \quad h^l[t] = \lambda_{post} h^l[t-1] + \Psi(u^l[t-1]).$$

因果迹和非因果迹：

$$e_{pre}^l[t] = \alpha_{pre} \Psi(u^l[t]) \otimes q^l[t], \quad e_{post}^l[t] = \alpha_{post} h^l[t] \otimes o^{l-1}[t].$$

内存： q 和 h 各 n 维，共 $\mathcal{O}(n)$ 。

4. 空间局部性（核心创新）

本地学习信号生成：

$$m^l[t] = B^{l\top} (f(B^l o^l[t]) - y^*). \quad (9)$$

四步分解：

- 1) 投影： $B^l o^l[t] \in \mathbb{R}^C$ ，将 n 维输出映射到 C 维任务空间。
- 2) 激活： $f(\cdot) = \text{softmax}$ （分类）或 identity （回归）。
- 3) 比较： $\pi^t - y^*$ ，与 one-hot 标签比较。
- 4) 反投影： $B^{l\top}$ 将误差映射回 n 维，得到 $m^l[t]$ 。

B^l 是固定随机矩阵，列向量为方波函数。优势：准正交性、硬件效率高、同步神经元活动。

5. 复杂度对比

Method	Memory	Time/step	Spatial Local?
BPTT	$\mathcal{O}(TLn)$	$\mathcal{O}(TLn^2)$	No
S-TLLR	$\mathcal{O}(Ln)$	$\mathcal{O}(Ln^2)$	No
TESS	$\mathcal{O}(Ln)$	$\mathcal{O}(LCn)$	Yes

$C \ll n$ ，因此 TESS 的 MAC 比 BPTT 减少约 n/C 倍。

XI. DISCUSSION QUESTIONS

- 1) **Exact vs. Approximate:** RTRL is exact but expensive. OTTT/TESS are approximate but efficient. When would you choose each?
- 2) **Biology-Inspired vs. Performance:** e-prop and S-TLLR incorporate biological mechanisms. Do these help performance, or are they just constraints?
- 3) **The Future of Local Learning:** TESS proves full locality is possible. Can it scale to ImageNet-level tasks?

【学生版讲稿】

讨论环节包含三个开放性问题。

第一，精确与近似的权衡。**RTRL** 提供精确梯度但内存为 $\mathcal{O}(n^3)$ ，适合小规模理论验证和基准测试。**OTTT/TESS** 提供近似梯度但内存为 $\mathcal{O}(n)$ ，适合大规模边缘部署和实时学习。实践中可采用“先精确后近似”的渐进策略。

第二，生物机制的作用。**e-prop** 的慢变量和 **S-TLLR** 的非因果项不仅仅是生物约束，它们确实提供了计算优势。慢变量解决了长期信用分配问题，非因果项在音频任务上带来了显著的性能提升。生物机制有时是约束，有时是启发。

第三，局部学习的未来。**TESS** 证明了完全局部学习是可行的，但能否扩展到 **ImageNet** 级别的任务仍需验证。可能的突破路径包括层次化设计（底层局部、顶层稀疏通信）和元学习初始化。

Key Takeaways — From RTRL to TESS:

- 1) **Shared Foundation:** All algorithms follow the same principle — a learning signal meets an eligibility trace. The difference lies in how each algorithm obtains these two signals.
- 2) **RTRL (1989):** The first online gradient algorithm. Exact but incredibly expensive — requires tracking sensitivity for every weight-output pair.
- 3) **e-prop (2020):** Added biological plausibility through slow ALIF neurons. These slow variables carry information across long time spans.
- 4) **OTTT (2022):** Discovered that most of BPTT’s temporal complexity can be safely ignored. The remaining part is just exponential decay. Memory becomes independent of sequence length.
- 5) **NDOT (2024):** Replaced OTTT’s fixed decay with a dynamic one based on actual neuron states. More accurate credit assignment with the same low memory.
- 6) **S-TLLR (2025):** Broke BPTT’s causality-only assumption. Added non-causal STDP terms. Different tasks prefer different settings — vision likes causality, audio likes synchrony.
- 7) **TESS (2025):** The ultimate goal achieved. No layer-to-layer error propagation. Each layer learns from locally generated signals. Fully parallelizable and hardware-friendly.

The big picture: Algorithms evolved from exact to biologically plausible, from expensive to efficient, from global to fully local.

参考文献

- [1] R. J. Williams and D. Zipser, “A learning algorithm for continually running fully recurrent neural networks,” *Neural computation*, vol. 1, no. 2, pp. 270–280, 1989.
- [2] G. Bellec, F. Scherr, A. Subramoney, E. Hajek, D. Salaj, R. Legenstein, and W. Maass, “A solution to the learning dilemma for recurrent networks of spiking neurons,” *Nature Communications*, vol. 11, no. 1, p. 3625, Jul. 2020. [Online]. Available: <https://doi.org/10.1038/s41467-020-17236-y>
- [3] M. Xiao, Q. Meng, Z. Zhang, D. He, and Z. Lin, “Online Training Through Time for Spiking Neural Networks,” in *Advances in Neural Information Processing Systems*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, Eds., vol. 35. Curran Associates, Inc., 2022, pp. 20 717–20 730. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2022/file/82846e19e6d42ebfd4ace4361def29ae-Paper-Conference.pdf
- [4] H. Jiang, G. D. Masi, H. Xiong, and B. Gu, “NDOT: Neuronal Dynamics-based Online Training for Spiking Neural...” [Online]. Available: <https://openreview.net/forum?id=eIF0QoBSFV>

- [5] M. P. E. Apolinario and K. Roy, "S-TLLR: STDP-inspired Temporal Local Learning Rule for Spiking Neural Networks," Oct. 2024, arXiv:2306.15220 [cs]. [Online]. Available: <http://arxiv.org/abs/2306.15220>
- [6] M. P. E. Apolinario, K. Roy, and C. Frenkel, "TESS: A Scalable Temporally and Spatially Local Learning Rule for Spiking Neural Networks," in *2025 International Joint Conference on Neural Networks (IJCNN)*, Jun. 2025, pp. 1–9, iSSN: 2161-4407. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/11227652>